Android MediaPlayer life cycle

The state transition diagram of MediaPlayer and characterization of its lifecycle, understanding this can help us to use MediaPlayer to consider more comprehensive, write the code more robust.

The state transition diagram clearly describe the states of MediaPlayer, also lists the main method call sequence, each method can only used in some specific conditions, if the use of MediaPlayer incorrect state will throw a IllegalStateException exception.

The Idle state: When using the new () method to create a MediaPlayer object and call its reset () method, the MediaPlayer object in the idle state. The two kind of method is one of the important difference is that: If in this state by using(getCurrentPosition(), getDuration(), getVideoHeight(), getVideoWidth(), setAudioStreamType(int), setLooping(boolean), setVolume(float, float), pause(), start(), stop(), seekTo(int), prepare() or prepareAsync())Methods(Equivalent to calling the incorrect time), Through the reset () method into the idle state would trigger OnErrorListener.onError(), And MediaPlayer will enter the Error state; if it is a newly created MediaPlayer object, Won't trigger (onError), can not enter the Error state.

The End state: Through the release () method can be in the End state, as long as the MediaPlayer object is no longer used, it should be as soon as possible through the release () method to release in order to release the soft hardware components, related resources, some of these resources is the only one of the (equivalent to the critical resource). If the MediaPlayer object in the End state, is not in any other state.

The Initialized state: This state is relatively simple, MediaPlayer calls setDataSource () method into the Initialized state, said at the time to play the file has been set up.

The Prepared state: After the initialization is done also by calling the prepare () or prepareAsync () method, the two method is a synchronization is asynchronous, only in the state of Prepared, it indicates that no errors of MediaPlayer so far, the file can be played.

The Preparing state: This state is better understood, and is the main prepareAsync (), if the asynchronous ready to complete, will trigger the OnPreparedListener.onPrepared (), and then enter the Prepared state.

The Started state: Apparently, MediaPlayer once ready, you can call start () method, so that the MediaPlayer is in the Started state, which indicates that MediaPlayer is in the process of playing the file. You can use the isPlaying (MediaPlayer) test is in the Started state. If the finished playing, and set the loop, then MediaPlayer will still be in the Started state, similar, if in the state of MediaPlayer call seekTo () or start () method can make MediaPlayer remains in the Started state.

The Paused state: Under the condition of Started MediaPlayer call pause () method can suspend MediaPlayer, To enter the Paused state, MediaPlayer call again suspended after start (MediaPlayer) can continue to play, Go to the Started state, The suspended state can call seekTo () method, This is not going to change the state of the.

The Stop state: Started or Paused state can call stop () to stop the MediaPlayer, and in the Stop state of MediaPlayer to re play, need through the prepareAsync () and prepare (Prepared) to return to the previous state restart can.

The PlaybackCompleted state: The file is normal finished playing, and is not set loop would enter the state, and will trigger the OnCompletionListener onCompletion () method. At this point you can call start () method to start playing the file, or stop (MediaPlayer), or seekTo () to re position the playback position.

The Error state: If for some reason the MediaPlayer error, will trigger the OnErrorListener.onError (MediaPlayer) event, this is in the Error state, timely capture and properly handle these errors is very important, can help us to release the software and hardware resources, but also can improve the user experience. Through setOnErrorListener (android.media.MediaPlayer.OnErrorListener) can be set to the listener. If the MediaPlayer is in the Error state, can call reset () to recover, makes the MediaPlayer back to Idle state.